

ESCUELA DE
INGENIERÍA ELÉCTRICA



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

Guía 9

Introducción al ESP32

Curso: EIE390 - Robótica e Inteligencia Artificial

Profesor: Guillermo Cid Ampuero

Objetivos

- **Objetivo 1:** Comprender la arquitectura, características y especificaciones técnicas del microcontrolador ESP32 para proyectos de Internet de las Cosas (IoT).
- **Objetivo 2:** Identificar la distribución y funcionalidad de los pines (Pinout), diferenciando entre GPIOs, pines analógicos, alimentación y de comunicación.
- **Objetivo 3:** Establecer las bases teóricas sobre los protocolos de comunicación disponibles (SPI, I2C, UART) para la integración de sensores y periféricos externos.

Repositorio del Curso

1 Introducción al ESP32

El ESP32 es un microcontrolador de alto rendimiento diseñado por Espressif Systems, ampliamente utilizado en el desarrollo de soluciones tecnológicas que requieren conectividad inalámbrica, procesamiento eficiente y bajo consumo energético. Equipado con conectividad Wi-Fi y Bluetooth de manera nativa, además de múltiples interfaces digitales y analógicas, se ha convertido en una plataforma clave para proyectos de automatización, monitoreo remoto, control industrial y aplicaciones del Internet de las Cosas (IoT).

Gracias a su arquitectura de doble núcleo y capacidad para ejecutar múltiples tareas de forma paralela, el ESP32 permite implementar sistemas complejos con una relación costo-beneficio notable. Su compatibilidad con entornos de desarrollo como Arduino IDE, PlatformIO y ESP-IDF lo convierte en una opción accesible tanto para principiantes como para usuarios avanzados.

Esta guía proporciona una base sólida para comprender el funcionamiento del ESP32, desde su configuración inicial hasta la programación de periféricos, comunicaciones inalámbricas y desarrollo de proyectos prácticos. Se promueve un enfoque progresivo, combinando fundamentos teóricos con ejercicios aplicados que favorecen el aprendizaje autónomo y el desarrollo de habilidades técnicas.

2 Especificaciones del ESP32

Característica	Especificación
Microcontrolador	ESP32 (Xtensa Dual-Core 32-bit LX6)
Frecuencia de CPU	Hasta 240 MHz
Memoria RAM	520 KB SRAM interna
Almacenamiento	Soporte para memoria Flash externa (4MB típico)
Voltaje de operación	3.0V a 3.3V (pines GPIO)
Voltaje de entrada (Vin)	5V (mediante conector USB o pin Vin)
Comunicación inalámbrica	Wi-Fi 802.11 b/g/n y Bluetooth 4.2 BLE
Cantidad de GPIO	Hasta 34 GPIO disponibles (dependiendo del modelo)
Interfaces	UART, SPI, I2C, I2S, CAN
ADC	12 bits, hasta 18 canales
DAC	2 canales de 8 bits
Pines Touch	Hasta 10 entradas táctiles capacitivas
PWM	Soporte para modulación de ancho de pulso
Consumo en Deep Sleep	Aproximadamente 10 150 μ A
Programación	USB-UART (cable microUSB)

Cuadro 1: Especificaciones principales del ESP32 DEVKIT V1

3 Pines del ESP32



Distribución de pines ESP32 DevKit V1

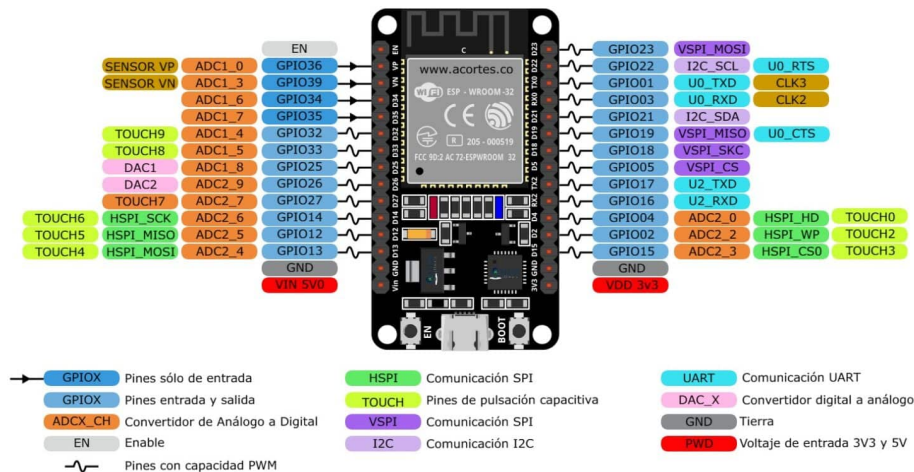


Figura 1: ESP32 y su distribución de pines (Pinout)

3.1 Pines de Alimentación

- 3V3: Salida de 3.3 V para alimentar dispositivos.
- GND: Tierra.
- VIN: Entrada de voltaje (de 5 a 12 V) utilizada para alimentar externamente el ESP32.

3.2 Pines GPIO (General Purpose Input/Output)

La placa proporciona numerosos pines de entrada/salida de propósito general (GPIO) que pueden utilizarse para diversas funciones digitales. Estos pines también admiten funcionalidades secundarias por hardware como PWM, I2C, SPI, entre otras.

3.3 Pines Analógicos

- **ADC (Analog to Digital Converter):** Permiten la lectura de señales analógicas. El ESP32 cuenta con 18 canales ADC de 12 bits de resolución. Algunos ejemplos de uso son: GPIO32 a GPIO39, GPIO0, GPIO2, GPIO4, y del GPIO12 al GPIO15.
- **DAC (Digital to Analog Converter):** Generan señales analógicas reales. Los pines disponibles son GPIO25 y GPIO26.

3.4 Pines RTC

Algunos GPIO también pueden funcionar como pines RTC (Real Time Clock), lo que permite su uso en modos de bajo consumo como el *deep sleep*. Incluyen: GPIO0, GPIO2, GPIO4, GPIO12 a GPIO15, y del GPIO32 al GPIO39.

3.5 Pines de Comunicación

■ SPI (Serial Peripheral Interface)

Protocolo de comunicación rápida y síncrona entre un maestro y uno o más esclavos, utilizando 4 líneas principales: MOSI, MISO, SCK y CS. Sirve para la comunicación entre sensores que requieren una transferencia de datos rápida con el microcontrolador, como pantallas o memorias flash.

- GPIO18: SCK (reloj)
- GPIO19: MISO (entrada)
- GPIO23: MOSI (salida)
- GPIO5: CS (chip select)

■ I2C (Inter-Integrated Circuit)

Se utiliza para la comunicación entre sensores o pantallas que requieran de una conexión sencilla y que permiten compartir varios dispositivos usando solo dos cables con el microcontrolador.

- GPIO22: SCL (reloj)
- GPIO21: SDA (datos)

■ UART (Serial)

Permite la comunicación entre el microcontrolador y otros dispositivos, intercambiando datos de manera sencilla mediante dos líneas: una para transmitir (TX) y otra para recibir (RX).

- UART0: GPIO1 (TX), GPIO3 (RX) — Usados habitualmente para la comunicación con el PC y flasheo.
- UART1/2: Pueden asignarse en otros pines mediante software, comúnmente usando GPIO9, GPIO10, GPIO16, GPIO17, entre otros.

3.6 Otros Pines Especiales

- EN: Pin de habilitación del chip. Al conectarlo a GND se reinicia el ESP32 (función Reset).
- GPIO0: Utilizado habitualmente para poner el ESP32 en modo de programación (BOOT) al arrancar.
- VP/VN: Corresponden a los pines GPIO36 (VP) y GPIO39 (VN). A menudo se utilizan para el sensor de efecto Hall interno o entradas analógicas.

4 Instalación y Configuración del Entorno de Desarrollo

Para programar el microcontrolador ESP32 utilizaremos el entorno de desarrollo de Arduino (Arduino IDE), el cual nos proporciona una interfaz amigable, un monitor serie integrado y compatibilidad con una enorme variedad de librerías.

4.1 1. Descarga e Instalación de Arduino IDE

1. Diríjase al sitio web oficial de Arduino: arduino.cc/en/software.
2. Descargue la versión más reciente de **Arduino IDE** (se recomienda la versión 2.x) correspondiente a su sistema operativo (Windows, macOS o Linux).
3. Ejecute el instalador descargado y siga las instrucciones en pantalla manteniendo la configuración por defecto.

4.2 2. Integración del ESP32 en Arduino IDE

Por defecto, el entorno de Arduino solo trae configuradas sus propias placas (Uno, Mega, Nano, etc.). Para poder compilar y transferir código al ESP32, debemos instalar el paquete oficial de tarjetas de Espressif.

Para poder realizar la integración de la ESP32 en la Arduino IDE realice el siguiente tutorial:

Problemas Tutorial

Installing ESP32 Add-on in Arduino IDE

4.3 3. Selección de la Placa y el Puerto de Comunicación

Una vez instalado el soporte de software, debemos indicarle al programa qué modelo físico de placa estamos usando y a través de qué puerto USB está conectada.

1. Conecte su ESP32 al computador utilizando el cable USB.
2. Vaya al menú **Herramientas Placa esp32** y seleccione el modelo de su placa. Generalmente, para las placas de desarrollo estándar se utiliza **DOIT ESP32 DEVKIT V1** o de manera genérica **ESP32 Dev Module**.
3. Vaya al menú **Herramientas Puerto** y seleccione el puerto de comunicación que haya aparecido al conectar la placa (por ejemplo, COM3 en Windows o /dev/cu.usbserial-... en macOS/Linux).

Problemas Frecuentes en el Laboratorio

- **El puerto no aparece:** Asegúrese de que el cable USB utilizado permita transferencia de datos. Muchos cables USB (como los de audífonos o cargadores genéricos) son **solo de carga**. Si el cable es de datos y aún no aparece, puede que necesite descargar el driver del conversor USB-Serial de su placa (usualmente el chip CP2102 o CH340).
- **Error al subir el código (Connecting... —...—):** Si la consola inferior arroja error al intentar subir el código, vuelva a darle al botón de subir y, apenas aparezca la palabra 'Connecting...', mantenga presionado el botón **BOOT** en la placa física hasta que comience el porcentaje de carga.

4.4 4. Instalación Manual del Controlador (Driver USB-Serial)

Si al conectar la placa el puerto COM no aparece o figura como 'Dispositivo desconocido' (CP2102 USB to UART Bridge Controller) con un triángulo amarillo, debe realizar la instalación manual:

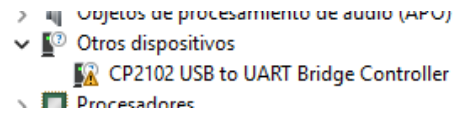


Figura 2: CP2102 Error

1. **Descarga:** Acceda a la web de Silicon Labs y descargue el *CP210x Universal Windows Driver* [Link de Descarga](#).
2. **Preparación:** Extraiga el contenido del archivo .zip en una carpeta conocida (ej. Escritorio).
3. **Actualización Manual:**
 - Abra el **Administrador de dispositivos**.
 - Haga clic derecho sobre el dispositivo con el triángulo amarillo.
 - Seleccione **Actualizar controlador** → **Examinar mi PC en busca de controladores**.
 - Seleccione la carpeta donde descomprimió los archivos y presione **Siguiente**.

5 Actividad 1: El ESP32 como Access Point (AP)

5.1 Objetivo

Comprender el concepto de Punto de Acceso (Access Point) y configurar el microcontrolador ESP32 para desplegar una red Wi-Fi local propia. El objetivo práctico es que cada estudiante logre conectar su teléfono celular de forma inalámbrica a la placa.

5.2 Instrucciones

1. **Concepto:** Lea el recuadro inferior para entender qué es un Access Point y cómo funciona en el ecosistema de Internet de las Cosas.
2. **Código:** En su entorno de desarrollo (Arduino IDE u otro compatible), cree un nuevo *sketch* y copie el código proporcionado a continuación.
3. **Configuración Única (¡Importante!):** Modifique las variables `ssid` y la configuración de la dirección `local_IP` en el código. **No todos deben usar la misma IP ni el mismo nombre de red.** Reemplace la 'X' en el código por su número de grupo o puesto de trabajo para evitar conflictos de red y cruce de conexiones en el laboratorio.
4. **Ejecución:** Conecte su ESP32 mediante el cable USB, compile y suba el código. Abra el Monitor Serie (configurado a 115200 baudios) para verificar que la red se ha iniciado correctamente y observar la IP asignada.
5. **Prueba:** Tome su teléfono celular, vaya a la configuración Wi-Fi, busque la red que acaba de crear su ESP32, ingrese la contraseña y verifique que el dispositivo logre conectarse.

Explicación: Access Point y Configuración de Red

¿Qué es un Access Point (AP)? Un Punto de Acceso crea una red de área local inalámbrica (WLAN). En este modo (conocido como SoftAP en el ESP32), el microcontrolador no se conecta a un router existente (como el de la universidad), sino que actúa como su propio 'router' básico. Permite que tu celular se conecte directamente a él.

Control de IP - `WiFi.softAPConfig()`: Por defecto, el ESP32 suele asignar la IP 192.168.4.1 a su red. En un laboratorio con múltiples equipos operando simultáneamente, utilizamos esta función para forzar una dirección IP estática única por grupo y evitar colisiones.

El Servidor Web (Puerto 80): Conectarse a la red Wi-Fi no es suficiente para visualizar información. Al incorporar la librería `WebServer`, le decimos al ESP32 que 'escuche' las peticiones que llegan por el puerto 80 (el estándar para HTTP). Cuando escribes la IP en tu navegador, el ESP32 intercepta esa petición y responde entregando el código HTML para mostrar en pantalla.

■ Código 1: ejercicio_1.ino

6 Actividad 2: Control Físico mediante Interfaz Web (Botones y LED)

6.1 Objetivo

Comprender la interacción entre el cliente (Navegador Web) y el servidor (ESP32) mediante la creación de una interfaz gráfica con botones en HTML que ejecuten acciones físicas reales (encender/apagar un LED) a través de peticiones HTTP.

6.2 Instrucciones

1. **Hardware:** Para esta actividad utilizaremos el pin `GPIO 2` del ESP32. En la mayoría de las placas de desarrollo, este pin está conectado internamente a un pequeño LED azul en la misma placa, por lo que puede realizar la prueba sin componentes externos. Si desea conectar un LED físico externo en una protoboard, conéctelo al pin 2 asegurándose de utilizar una resistencia limitadora de corriente (ej. $330\ \Omega$).
2. **Código:** En su entorno de desarrollo, cree un nuevo *sketch* e implemente el código correspondiente a la Actividad 2. Al igual que en la actividad anterior, **modifique el SSID y la IP** por los asignados a su grupo.
3. **Ejecución:** Compile y suba el código a su ESP32. Abra el Monitor Serie para confirmar que el servidor se ha iniciado correctamente.
4. **Prueba:** Conecte su celular a la red Wi-Fi de su ESP32. Abra el navegador, ingrese la dirección IP asignada y presione los botones en la pantalla para verificar que el LED físico responde en tiempo real.

6.3 Explicación: ¿Cómo funciona un Botón Web?

Para que un botón táctil en la pantalla del celular logre encender un componente electrónico en el laboratorio, necesitamos coordinar dos mundos: el *Frontend* (lo que ve el usuario) y el *Backend* (la lógica de control).

1. El Frontend (HTML en el Navegador)

En nuestro código de C++, construimos y enviamos la siguiente línea de HTML al navegador:

```
<button onclick='window.location.href='/encender''>Encender LED</button>
```

- `<button>...</button>`: Son las etiquetas estándar de HTML que le dicen al navegador 'dibuja un botón aquí'.
- `onclick`: Es un evento de JavaScript. Le indica al celular que debe estar atento al instante en que el usuario toque ese botón.
- `window.location.href='/encender'`: Es la acción a ejecutar. Al presionar el botón, le ordenamos al navegador que viaje a una nueva ruta específica llamada `/encender` dentro de nuestro servidor (por ejemplo, navegando internamente a `192.168.10.1/encender`).

2. El Backend (C++ en el ESP32)

Cuando el navegador del celular intenta visitar esa nueva ruta `/encender`, el ESP32 debe estar preparado para 'atrapar' esa visita. Para ello utilizamos la siguiente instrucción en el `setup()`:

```
server.on('/encender', handleEncender);
```

- `server.on()`: Es el puente entre el mundo web y el hardware. Vincula una ruta de internet (`'/encender'`) con una función específica escrita en C++ (`handleEncender`).
- `digitalWrite(ledPin, HIGH)`: Finalmente, dentro de la función `handleEncender()`, el microcontrolador ejecuta la instrucción física enviando 3.3V al pin, cerrando el ciclo. Inmediatamente después, el servidor vuelve a enviarle al celular la página web actualizada.

▪ Código 2: ejercicio_2.ino

7 Actividad 3 (Desafío): Control de Múltiples Actuadores

7.1 Objetivo

Escalar la arquitectura del servidor web añadiendo nuevas rutas (endpoints) y elementos en la interfaz gráfica para controlar de forma independiente un segundo actuador físico: un zumbador o *Buzzer*.

7.2 Instrucciones

1. **Hardware:** Conecte un Buzzer activo a su ESP32. Conecte el pin positivo (generalmente el pin más largo) al pin digital **GPIO 4** y el pin negativo a cualquiera de los pines **GND** de la placa.
2. **Análisis de Código:** Copie el código de la Actividad 2. Para cumplir con este desafío, deberá identificar y duplicar la lógica utilizada para el LED, pero ahora aplicándola al Buzzer.
3. **Modificaciones Requeridas:**
 - Declare el pin del Buzzer y configúrelo como salida en el `setup()`.
 - En la función `generarHTML()`, añada el código HTML para mostrar el estado del Buzzer y dos nuevos botones (por ejemplo, con las rutas `/buzzer_on` y `/buzzer_off`).
 - Cree las dos nuevas funciones controladoras (ej. `handleBuzzerOn` y `handleBuzzerOff`) que modifiquen el estado físico del pin 4 usando `digitalWrite()`.
 - Vincule estas nuevas rutas con sus respectivas funciones utilizando `server.on()` dentro del `setup()`.
4. **Prueba:** Suba el código modificado, conéctese a la red Wi-Fi y verifique desde el navegador que puede encender y apagar tanto el LED como el Buzzer de manera independiente.

Tip de Diseño: Creación de Paneles

Al agregar más botones y textos, la interfaz puede volverse desordenada. En HTML, la etiqueta `<div>` (división) se utiliza como un contenedor invisible para agrupar elementos relacionados.

Si le asignamos una clase CSS a este contenedor (ej. `<div class='panel'>`), podemos darle estilo de recuadro en la sección `<style>` de nuestro código. Así, logramos separar visualmente los controles del LED de los controles del Buzzer:

```
// Inicio del panel
html += "<div class='panel'>";
html += "<h2>Buzzer Activo</h2>";
// ... (Aquí van los botones y textos del buzzer) ...
html += "</div>"; // Fin del panel
```

Pista para el Desafío: Escalabilidad de la API

En el desarrollo de APIs, cada acción específica requiere su propia ruta o *endpoint*. Al igual que creamos las rutas `/encender` y `/apagar` para el LED, el servidor necesita rutas únicas para el Buzzer para no confundir las órdenes.

Al agregar botones en HTML, puede reutilizar las clases CSS existentes (`btn-on` y `btn-off`) para mantener el diseño consistente. Solo necesita cambiar el texto del botón y el atributo `onclick` para que apunte a las nuevas rutas.

■ Código 3: ejercicio_3_desafio.ino

8 Referencias

1 [Link 1](#)